CONSOLE VIRTUELLE POUR MiSTer · DE10-NANO

MONOCORE-1

MANUEL DU PROGRAMMEUR



Écrivez vos propres jeux · Tutoriels pas à pas · Référence complète

ÉDITION 1 · OCTOBRE 2026

Sommaire

1 La MONOCORE-1 en un coup d'oeil 3
2 Installer l'atelier 5
3 Premier programme : BONJOUR 7
4 Petit cours de C 9
5 La manette 12
6 Les sprites 14
7 Le son 17
8 Le décor 20
9 Projet : ATTRAPE-ÉTOILES 22
10 Sous le capot 26
A La bibliothèque easy 28
B Carte mémoire et registres 30
C Limites et dépannage 32

Avant-propos

Bienvenue ! Vous avez entre les mains le manuel de la **MONOCORE-1**, une console de jeu qui n'a jamais existé dans le commerce : elle vit dans le circuit programmable (FPGA) de votre MiSTer. Elle a été conçue pour accueillir des adaptations de jeux d'arcade des années 1978 à 1995, mais aussi, et surtout, **vos propres jeux**.

Comme les ordinateurs familiaux de notre enfance, elle s'allume et attend qu'on lui dise quoi faire. Ce manuel vous apprend à le lui dire. **Aucune connaissance préalable n'est nécessaire** : nous partirons d'un écran vide pour arriver, au chapitre 9, à un jeu complet avec un titre, un score, des vies, de la musique et des explosions.

Chaque exemple de ce manuel est un vrai programme, fourni dans le dossier `manual/examples`. Les images d'écran ont été prises en le faisant tourner : ce que vous voyez est ce que vous obtiendrez.

Une langue pour le code

Comme dans presque tous les langages de programmation, les noms utilisés dans le code (fonctions, variables, fichiers, réglages) sont en **anglais** : `print` pour écrire, `wait_frame` pour attendre l'image suivante... Chaque nom est expliqué la première fois qu'il apparaît, et l'annexe A les traduit tous. Les textes affichés à l'écran, eux, sont en français.

Comment lire ce manuel

- Les chapitres 1 à 3 installent l'atelier et font tourner un premier programme. Lisez-les dans l'ordre.
- Les chapitres 4 à 8 présentent une notion chacun : le langage C, la manette, les sprites, le son, le décor.
- Le chapitre 9 assemble tout dans un jeu. Le chapitre 10 ouvre le capot pour les curieux.
- Les annexes servent de référence : gardez-les sous la main pendant que vous programmez.

Bon courage, et amusez-vous bien !

1

La MONOCORE-1 en un coup d'oeil

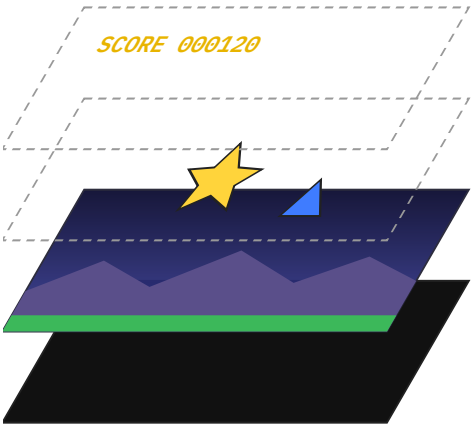
Ce qu'il y a dans la boîte, et comment l'image et le son sont fabriqués.

La MONOCORE-1 est une console complète : un processeur, une mémoire, un circuit vidéo, un circuit sonore et deux manettes. Un programme de jeu, une **ROM**, est un fichier `.mnc` que l'on charge depuis le menu du MiSTer ou que l'on glisse dans le simulateur.

Élément	Caractéristiques
Processeur	RISC-V 32 bits à 50 MHz, environ 47 millions d'instructions par seconde
Mémoire	128 Ko de mémoire de travail, 32 Ko de mémoire sauvegardée, ROM jusqu'à 32 Mo
Image	320 x 240 pixels par défaut (de 224 x 192 à 384 x 288), 60 images par seconde, 4096 couleurs à l'écran parmi 32 768
Plans	4 plans de tuiles qui défilent, chacun dans sa direction
Sprites	512 objets mobiles, agrandis ou réduits à volonté, retournés, jusqu'à 64 par ligne
Son	8 voix d'échantillons (dont le format ADPCM des bornes d'arcade), 8 générateurs de notes et de bruit, stéréo 48 kHz
Commandes	2 manettes : 4 directions, 6 boutons, START et COIN (pièce)

Comment se forme l'image

L'écran est construit 60 fois par seconde, ligne par ligne, en superposant plusieurs couches. Avec la bibliothèque **easy** (« facile ») que nous utiliserons dans ce manuel, il y en a quatre :



4. le texte (plan 0)
toujours devant

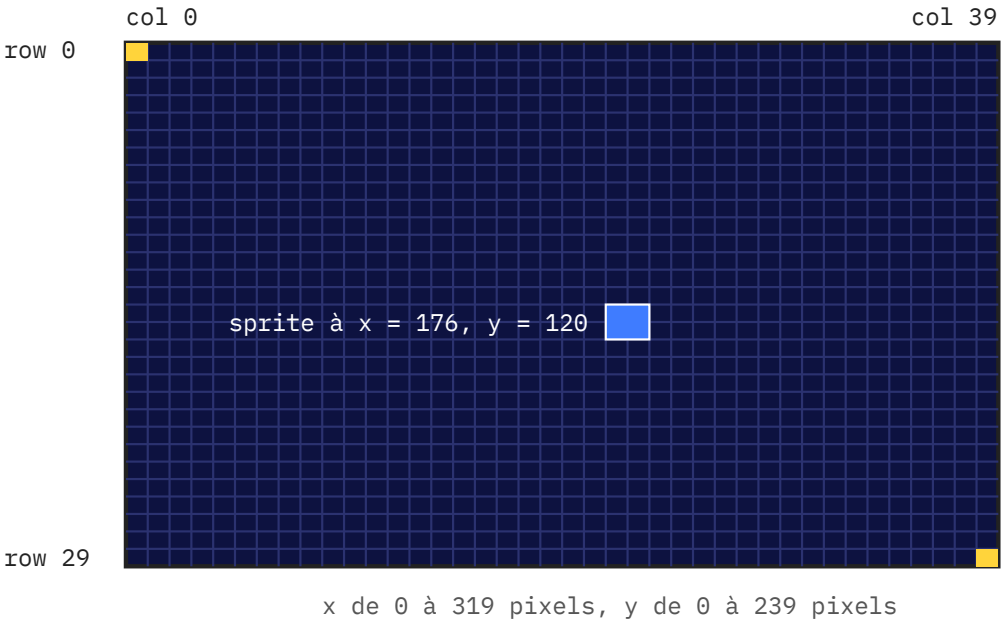
3. les sprites
64 objets mobiles de 16 x 16

2. le décor (plan 1)
tuiles de 16 x 16, il défile

1. la couleur de fond
là où rien n'est dessiné

L'image est un empilement : ce qui est dessiné au-dessus cache ce qui est dessous.

Pour placer les choses à l'écran, on utilise deux façons de compter. Le texte se range dans une grille de cases de 8 x 8 pixels : 40 colonnes et 30 lignes. Les sprites, eux, se placent au pixel près : x va de 0 (à gauche) à 319 (à droite), y de 0 (en haut) à 239 (en bas). Attention, y grandit vers le **bas**, comme sur toutes les consoles.



Le texte se place en colonnes et lignes (une case = 8 x 8 pixels), les sprites au pixel près.

Le saviez-vous ?

Une image de la MONOCORE-1 compte 264 lignes, dont 240 visibles. Pendant les 24 lignes invisibles, la **pause verticale**, rien n'est dessiné : c'est le bon moment pour préparer l'image suivante. La fonction `wait_frame()` (« attendre l'image ») attend précisément ce moment.

2

Installer l'atelier

Trois outils gratuits, un simulateur, et la console est prête à recevoir vos programmes.

Un programme MONOCORE-1 s'écrit en **langage C** dans un simple éditeur de texte, puis il est transformé en ROM par l'outil `build.py` (« construire »). Il faut pour cela :

1. **LLVM** (le compilateur `clang` et ses amis) : il sait produire du code pour le processeur RISC-V de la console ;
2. **Python 3** avec la bibliothèque **Pillow**, pour convertir vos dessins et vos sons ;
3. un **éditeur de texte** : n'importe lequel convient (Bloc-notes, TextEdit, Geany, VS Code…).

SOUS LINUX (DEBIAN, UBUNTU, RASPBERRY PI OS…)

```
sudo apt install clang lld llvm python3 python3-pil
```

SOUS MACOS

```
brew install llvm lld python
pip3 install pillow
export PATH="$(brew --prefix llvm)/bin:$PATH"
```

La dernière ligne ajoute les outils LLVM au chemin de recherche des commandes ; ajoutez-la à votre fichier `~/.zshrc` pour ne pas avoir à la retaper.

SOUS WINDOWS

Le plus simple est d'installer **WSL** (le Linux intégré à Windows : dans une invite de commandes, tapez `wsl --install`, redémarrez) puis de suivre les instructions Linux ci-dessus. On peut aussi installer LLVM (llvm.org) et Python (python.org) directement, puis `pip install pillow`.

VÉRIFIER QUE TOUT EST PRÊT

TERMINAL — deux lignes doivent s'afficher

```
clang -print-targets | grep riscv32
python3 -c "import PIL; print('Pillow OK')"
```

Le dossier de la console

Le projet MONOCORE-1 est un dossier qui contient, entre autres :

Dossier	Contenu
`sdk/`	le kit de développement : `easy.h` , `monocore.h` , l'outil `build.py` , la police de caractères
`manual/examples/`	tous les programmes de ce manuel, prêts à construire
`dist/monocore.html`	le simulateur : une page web qui fait tourner les ROM sur votre ordinateur
`mister/`	le core pour le MiSTer

Le simulateur

Ouvrez `dist/monocore.html` dans votre navigateur (Firefox, Chrome, Edge ou Safari). Cliquez sur **Mettre sous tension** : la ROM de test démarre. Pour essayer vos programmes, cliquez sur **Charger une ROM…** ou glissez un fichier `.mnc` sur l'écran. Le simulateur se joue au clavier ou avec une manette USB :

Commande	Joueur 1	Joueur 2
Directions	flèches	I J K L
Boutons 1 à 3	Z X C	N M ,
Boutons 4 à 6	A S D	H U O
START	1 ou Entrée	2
COIN (pièce)	5	6

Sur le côté, le **moniteur** montre la charge du processeur, le nombre de sprites par ligne, les voix sonores actives, et une **console de debug** qui affiche ce que votre programme écrit dans `DEBUG_OUT` , bien pratique pour chercher une erreur.

La console MiSTer

Pour jouer sur la vraie console, copiez le core `MONOCORE.rbf` dans le dossier `_Console` de la carte SD du MiSTer, et vos ROM dans `games/MONOCORE` . Dans le menu du core (touche F12), choisissez **Charger une ROM**. Une ROM nommée `boot.rom` dans ce dossier se lance toute seule.

> **Attention**
>
> Le MiSTer doit être équipé d'une **barrette SDRAM** : la ROM du jeu est chargée dedans.

3

Premier programme : BONJOUR

Six lignes de C, une commande, et la console vous salue.

Tout programme vit dans son propre dossier. Celui-ci s'appelle `01_hello` et contient deux fichiers. Le premier, `project.txt`, décrit le projet : le nom du fichier à produire (`name`), le titre inscrit dans la ROM (`title`) et les fichiers du programme (`sources`).

LISTING project.txt

```
name    = hello
title   = BONJOUR
sources = main.c
```

Le second, `main.c`, est le programme lui-même :

LISTING main.c

```c
#include "easy.h"

int main(void) {
  init();
  background(0, 0, 120);
  print(14, 12, "BONJOUR !");
  print(8, 16, "JE SUIS LA MONOCORE-1.");
  for (;;) wait_frame();
}
```

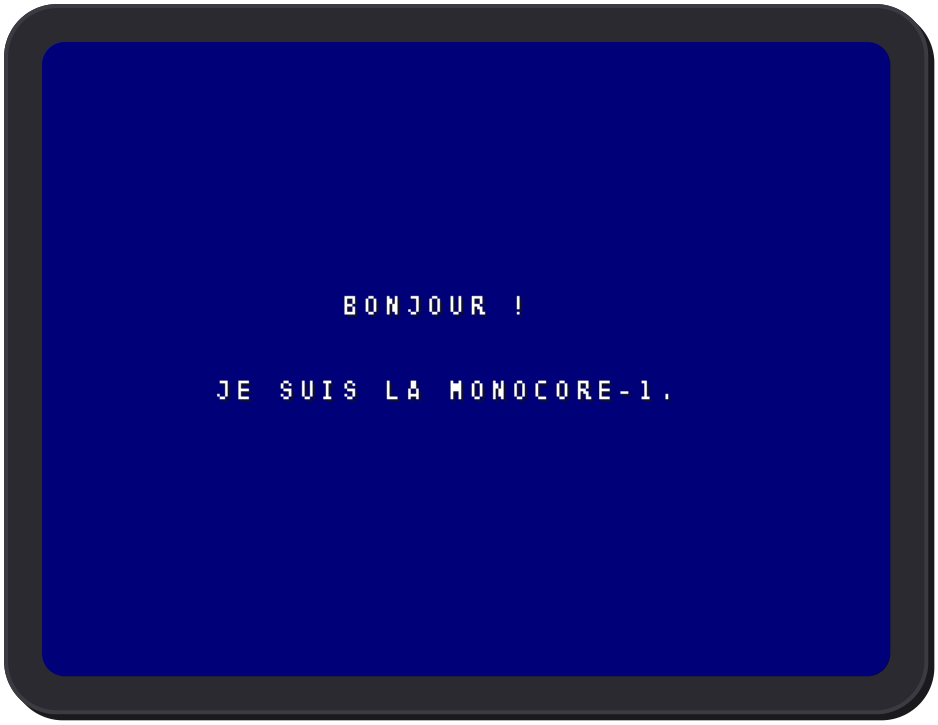
Construire la ROM

Ouvrez un terminal dans le dossier du programme, et lancez `build.py` en indiquant où il se trouve :

```
cd manual/examples/01_hello
python3 ../../../sdk/build.py

ROM ready: .../01_hello/build/hello.mnc
```

Le fichier `build/hello.mnc` est votre ROM. Chargez-la dans le simulateur :



BONJOUR en 320 x 240 : le texte blanc sur le fond bleu.

Ligne par ligne

Ligne	Ce qu'elle fait
1	`#include "easy.h"` donne accès à la bibliothèque easy : toutes les fonctions de ce manuel.
3	`int main(void)` : c'est ici que le programme commence. Tout ce qui est entre les accolades `{ }` s'exécute dans l'ordre.
4	`init()` (« initialiser ») prépare la console : police, couleurs, écran allumé. À appeler une fois, en premier.
5	`background(0, 0, 120)` choisit la couleur du fond : rouge 0, vert 0, bleu 120 (sur 255).
6 et 7	`print(colonne, ligne, "texte")` (« imprimer ») écrit un texte dans la grille de 40 x 30 cases.
8	`for (;;)` répète sans fin ce qui suit. Un programme de console ne s'arrête jamais : ici, il attend image après image avec `wait_frame()`.

À vous de jouer

- Changez le texte, sa position, la couleur du fond, puis reconstruisez la ROM.
- Les accents sont acceptés mais affichés sans accent : la police est celle des années 80, en majuscules de préférence.
- Une erreur ? `build.py` indique le fichier et la ligne. Le plus souvent, il manque un point-virgule `;` en fin de ligne.

4

Petit cours de C

Variables, conditions et boucles : de quoi donner vie à l'écran.

Le C est le langage dans lequel ont été écrits une grande partie des jeux et des systèmes depuis les années 80. Nous n'en utiliserons qu'une petite partie, présentée ici sur un exemple qui affiche les huit encres de texte, compte les secondes et fait clignoter un message.

LISTING 02_colors/main.c

```c
#include "easy.h"

int main(void) {
  init();
  background(20, 0, 60);                    // a very dark purple

  print_ink(12, 3, "LES HUIT ENCRES", 1);    // ink 1: yellow
  for (int n = 0; n < 8; n++) {              // n takes the values 0, 1, ... up to 7
    print_ink(12, 7 + n * 2, "ENCRE NUMERO", n);
    print_num(25, 7 + n * 2, n, 1, n);
  }

  int seconds = 0;
  for (;;) {                                 // forever
    wait_frame();
    if (frame_count % 60 == 0) seconds = seconds + 1;   // 60 frames = 1 second
    print(9, 25, "SECONDES :");
    print_num(20, 25, seconds, 4, 2);

    // blinking text: on for half a second, off for half a second
    if ((frame_count / 30) % 2 == 0) print_ink(12, 27, "APPUIE SUR START", 6);
    else                             print    (12, 27, "                ");

    if (pressed(1, START)) background(random(256), random(256), random(256));
  }
}
```



Les huit encres, le compteur de secondes et le message qui clignote.

Les variables

Une **variable** est une case de mémoire qui porte un nom et contient un nombre. `int seconds = 0;` crée la variable `seconds` (« secondes ») et y range 0. Ensuite, `seconds = seconds + 1;` lit la valeur, ajoute 1 et range le résultat. `int` veut dire « nombre entier » (de -2 milliards à +2 milliards).

La bibliothèque fournit une variable toute prête : `frame_count` (« nombre d'images »), qui compte les images depuis le démarrage. Comme il y en a 60 par seconde, `frame_count % 60 == 0` (le reste de la division par 60 vaut zéro) est vrai une fois par seconde.

Les conditions

`if (condition) action;` n'exécute l'action que si la condition est vraie. On peut ajouter `else autre_action;` pour le cas contraire. Les comparaisons s'écrivent `==` (égal, avec deux signes !), `!=` (différent), `<`, `>`, `<=`, `>=`. On combine avec `&&` (et) et `||` (ou).

Les boucles

`for (int n = 0; n < 8; n++)` répète ce qui suit en faisant prendre à `n` les valeurs 0, 1, 2... jusqu'à 7. `n++` veut dire « ajouter 1 à n ». La forme `for (;;)`, sans rien, répète pour toujours : c'est la **boucle principale** de tout jeu, qui tourne une fois par image.

Les fonctions

`print_ink(...)`, `background(...)` ou `random(256)` sont des **fonctions** : des morceaux de programme qui ont un nom, reçoivent des valeurs entre parenthèses et parfois en rendent une. `random(256)` (« au hasard ») rend un nombre entre 0 et 255. Vous écrirez vos propres fonctions au chapitre 9.

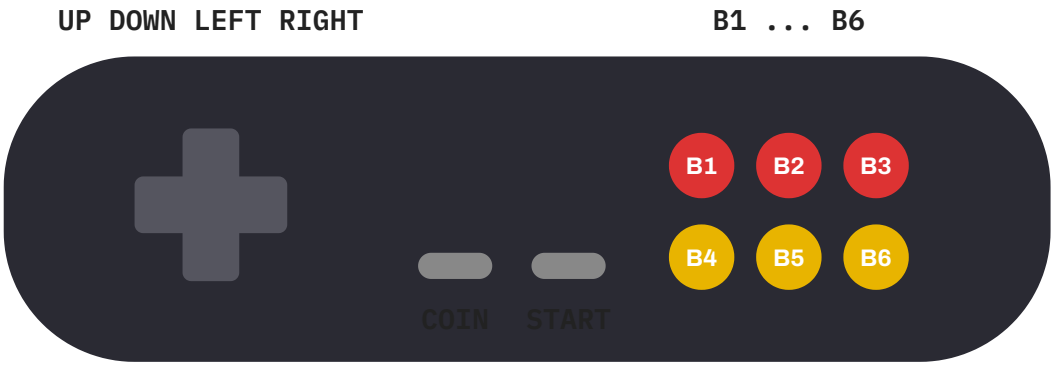
La recette de tout jeu

```
int main(void) {
  init();
  // 1. set up: colours, pictures, map, variables
  for (;;) {
    wait_frame();
    // 2. read the joypad
    // 3. move the world, check collisions
    // 4. draw: sprites, score
  }
}
```

Préparer, puis pour chaque image : lire la manette, faire bouger le monde, afficher.

La manette

Savoir ce que fait le joueur, image après image.



Les noms des commandes, tels qu'on les écrit dans un programme.

Deux fonctions suffisent pour tout savoir des manettes :

Fonction	Rend vrai quand…	Pour…
`held(1, LEFT)`	le bouton est enfoncé en ce moment (*held* : tenu)	les déplacements, le tir automatique
`pressed(1, B1)`	le bouton vient d'être enfoncé, une seule fois par appui (*pressed* : appuyé)	sauter, valider un menu, compter

Le premier nombre est le joueur : 1 ou 2. Les manettes sont lues une fois par image, par `wait_frame()`. Voici un programme d'essai qui affiche l'état de chaque commande :

LISTING 03_joypad/main.c

```c
#include "easy.h"

// a label and a button for each line of the display
static const char *labels[12] = {
  "HAUT", "BAS", "GAUCHE", "DROITE", "BOUTON 1", "BOUTON 2",
  "BOUTON 3", "BOUTON 4", "BOUTON 5", "BOUTON 6", "START", "PIECE"};
static const uint32_t buttons[12] = {
  UP, DOWN, LEFT, RIGHT, B1, B2, B3, B4, B5, B6, START, COIN};

int main(void) {
  init();
  background(0, 40, 30);
  print_ink(10, 1, "ESSAI DE LA MANETTE", 1);
  int presses = 0;

  for (;;) {
    wait_frame();
```

```
    for (int i = 0; i < 12; i++) {
      print(10, 4 + i * 2, labels[i]);
      if (held(1, buttons[i])) print_ink(22, 4 + i * 2, "ENFONCE", 3);
      else                     print_ink(22, 4 + i * 2, ".......", 7);
    }
    if (pressed(1, B1)) presses++;          // once per press, even if the button stays
down
    print(6, 28, "APPUIS SUR LE BOUTON 1 :");
    print_num(31, 28, presses, 3, 2);
  }
}
```



Ici, HAUT, DROITE et le bouton 2 sont enfoncés.

Ce programme utilise deux **tableaux** : `labels[12]` contient douze textes, `buttons[12]` douze boutons. `labels[0]` est le premier, `labels[11]` le dernier. La boucle `for` les parcourt tous avec la même variable `i`.

6

Les sprites

Des objets qui bougent : votre héros, les ennemis, les bonus.

Dessiner une planche

Les dessins des sprites se rangent dans une **planche** (*sprite sheet*) : une image PNG de **16 pixels de large**, où les dessins de 16 x 16 sont posés les uns sous les autres. Le premier porte le numéro 0, le suivant le numéro 1, et ainsi de suite. Dessinez-la avec n'importe quel logiciel de dessin (Aseprite, LibreSprite, GIMP, Paint...) en respectant deux règles :

- **15 couleurs au plus** pour toute la planche ;
- le **transparent** (les pixels effacés de l'image) laisse voir ce qui est derrière le sprite.



N°	Nom dans le code	Dessin
0	`SHIP`	vaisseau
1	`STAR`	étoile
2	`STAR2`	étoile qui scintille
3	`BOMB`	bombe
4	`EXPLOSION`	explosion
5	`HEART`	cœur

La planche des exemples, agrandie huit fois. Elle est dans `04_sprites/sheet.png`.

Il suffit de la nommer dans `project.txt` (`sprites = sheet.png`) : `build.py` la convertit et la rend disponible sous les noms `sheet_chr` (les dessins) et `sheet_pal` (les couleurs).

Afficher et déplacer

LISTING 04_sprites/main.c

```
#include "easy.h"

extern const uint8_t sheet_chr[], sheet_pal[];   // built from sheet.png

// picture numbers in the sheet
#define SHIP   0
#define STAR   1
```

```c
#define STAR2  2

int main(void) {
  init();
  background(0, 0, 40);
  load_sprites(sheet_chr, sheet_pal, 8);
  print_ink(3, 1, "MANETTE:BOUGER  B1:MIROIR  B2:TAILLE", 2);

  int x = 152, y = 120;          // ship position
  int flipped = 0, big = 0;

  for (;;) {
    wait_frame();
    if (held(1, LEFT))  x = x - 2;
    if (held(1, RIGHT)) x = x + 2;
    if (held(1, UP))    y = y - 2;
    if (held(1, DOWN))  y = y + 2;
    if (pressed(1, B1)) { flipped = !flipped; flip_sprite(0, 0, flipped); }
    if (pressed(1, B2)) { big = !big;         scale_sprite(0, big ? 200 : 100); }
    sprite(0, x, y, SHIP);

    // five twinkling stars: the picture changes every 10 frames
    for (int i = 0; i < 5; i++) {
      int picture = ((frame_count / 10 + i) % 2) ? STAR : STAR2;
      sprite(1 + i, 30 + i * 60, 50, picture);
    }
  }
}
```



Le vaisseau agrandi deux fois (bouton 2), et les étoiles qui scintillent.

Fonction	Effet
`load_sprites(chr, pal, 8)`	choisit la planche et range ses couleurs dans la palette 8
`sprite(n, x, y, picture)`	affiche le sprite n (0 à 63) à la position x, y avec le dessin choisi
`hide_sprite(n)`	fait disparaître le sprite n
`flip_sprite(n, h, v)`	le retourne horizontalement (h = 1) et/ou verticalement (v = 1)
`scale_sprite(n, 200)`	l'agrandit (200 = double, 50 = moitié)

Fonction	Effet
`collide(x1, y1, x2, y2)`	vrai si deux sprites de 16 x 16 se touchent

Animer

Pour animer un sprite, on change de dessin régulièrement. `(frame_count / 10) % 2` vaut 0 pendant 10 images, puis 1 pendant 10 images, etc. : de quoi alterner deux dessins six fois par seconde. Avec `% 4`, on fait tourner quatre dessins.

Les limites

On peut afficher jusqu'à **64 sprites sur une même ligne** de l'écran. Au-delà, les derniers ne sont pas dessinés sur cette ligne : le moniteur du simulateur l'indique en rouge.

7

Le son

Des bips, des bruits, des mélodies et des échantillons.

La MONOCORE-1 a **8 canaux** de notes (numérotés de 0 à 7), qui jouent ensemble, et **8 voix d'échantillons** pour les sons enregistrés.

Fonction	Effet
`tone(channel, 440, 150)`	joue un son carré de 440 hertz (le LA du diapason) au volume 150 sur 255
`noise(channel, 120)`	un souffle : explosions, vagues, caisse claire
`quiet(channel)`	fait taire le canal
`play(channel, tune, 15, 100)`	joue une mélodie en boucle, toute seule
`stop(channel)`	arrête la mélodie
`play_sample(voice, data, count, rate, volume)`	joue un son enregistré (fichier WAV)

LISTING 05_sound/main.c

```c
#include "easy.h"

// "Au clair de la lune", one note per beat
static const char *moonlight =
  "C C C D E - D - C E D D C - - . "
  "C C C D E - D - C E D D C - - . ";

int main(void) {
  init();
  background(40, 0, 0);
  print_ink(13, 3, "LE PETIT PIANO", 1);
  print(4, 8,  "BOUTONS 1 A 6 : DO RE MI FA SOL LA");
  print(4, 10, "HAUT          : SI");
  print(4, 12, "BAS           : BRUIT");
  print(4, 14, "START         : MUSIQUE OUI / NON");

  static const int notes[6] = {262, 294, 330, 349, 392, 440};   // C D E F G A, in hertz
  static const uint32_t keys[6] = {B1, B2, B3, B4, B5, B6};
  int playing = 0;

  for (;;) {
    wait_frame();
    int frequency = 0;
    for (int i = 0; i < 6; i++)
      if (held(1, keys[i])) frequency = notes[i];
    if (held(1, UP)) frequency = 494;                    // B
    if (frequency) tone(0, frequency, 150); else quiet(0);
    if (held(1, DOWN)) noise(1, 120); else quiet(1);

    if (pressed(1, START)) {
      playing = !playing;
      if (playing) play(2, moonlight, 15, 100);
      else         stop(2);
```

```
    }
    print_ink(12, 20, playing ? "MUSIQUE : OUI" : "MUSIQUE : NON", playing ? 3 : 7);
  }
}
```



Le petit piano : chaque bouton joue une note.

Écrire une mélodie

Une mélodie est un texte où chaque mot est un **temps**. Les notes s'écrivent avec les lettres de la notation anglaise :

DO	Ré	MI	FA	SOL	LA	SI
`C`	`D`	`E`	`F`	`G`	`A`	`B`

écriture	Signification
`C D E F G A B`	les notes, dans l'octave 4 (celle du LA à 440 Hz)
`C5 G3`	un chiffre après la note change d'octave : C5 est le DO aigu suivant
`F# C#5`	un # (dièse) monte la note d'un demi-ton
`-`	prolonge la note précédente d'un temps
`.`	un temps de silence

Le troisième nombre de `play()` règle le tempo : le nombre d'images par temps. 15 images, c'est 4 temps par seconde (240 à la minute) ; 30, deux fois plus lent. Pour jouer plusieurs voix en même temps (une mélodie et une basse), lancez deux mélodies sur deux canaux.

Les sons enregistrés

Un fichier WAV nommé dans `project.txt` (`sounds = boom.wav`) devient trois noms : `boom_data` (les échantillons), `boom_count` (leur nombre) et `boom_rate` (leur fréquence). On le joue avec :

```
extern const int8_t boom_data[];
extern const uint32_t boom_count, boom_rate;
...
play_sample(0, boom_data, boom_count, boom_rate, 230);
```

Bonne pratique

Gardez des sons courts et en 8 000 à 22 050 Hz : ils occupent moins de place dans la ROM et sonnent très « arcade ». Un son de 16 000 Hz pendant une demi-seconde occupe 8 Ko.

Le décor

Un grand paysage en tuiles, qui défile derrière les sprites.

Le décor (*map*) est une grande image découpée en **tuiles de 16 x 16 pixels**. Les tuiles identiques ne sont stockées qu'une fois : un ciel uni ou un sol répétitif ne coûtent presque rien. Le décor peut mesurer jusqu'à **1024 x 1024 pixels** et il boucle : en défilant au-delà du bord droit, on retrouve le bord gauche.

- Dessinez une image dont la largeur et la hauteur sont des multiples de 16, en **15 couleurs** au plus ;
- nommez-la dans `project.txt` : `maps = landscape.png` ;
- le programme reçoit `landscape_chr`, `landscape_pal`, `landscape_map` (la carte des tuiles), `landscape_width` et `landscape_height` (largeur et hauteur en tuiles).

LISTING 06_scroll/main.c

```c
#include "easy.h"

extern const uint8_t sheet_chr[], sheet_pal[];
extern const uint8_t landscape_chr[], landscape_pal[], landscape_map[];
extern const uint32_t landscape_width, landscape_height;     // in 16 x 16 tiles

int main(void) {
  init();
  load_sprites(sheet_chr, sheet_pal, 8);
  load_map(landscape_chr, landscape_pal, landscape_map, landscape_width,
landscape_height, 9);
  print_ink(2, 1, "GAUCHE / DROITE : VITESSE", 1);

  int camera = 0;            // screen position in the map, in pixels
  int speed = 1;
  int y = 100;

  for (;;) {
    wait_frame();
    if (pressed(1, RIGHT) && speed < 6)  speed++;
    if (pressed(1, LEFT)  && speed > -6) speed--;
    if (held(1, UP)   && y > 20)  y -= 2;
    if (held(1, DOWN) && y < 160) y += 2;

    camera = camera + speed;
    scroll_map(camera, 0);               // the map is 1024 pixels wide and wraps around
by itself

    sprite(0, 152, y, 0);
    print(2, 28, "VITESSE :");
    print_num(12, 28, speed, 2, 2);
  }
}
```



Le paysage de 1024 pixels défile derrière le vaisseau.

`scroll_map(x, y)` (« faire défiler le décor ») indique quel point du décor apparaît dans le coin en haut à gauche de l'écran. En augmentant `x` à chaque image, le décor glisse vers la gauche : on a l'impression d'avancer vers la droite.

Le saviez-vous ?

Les jeux des années 80 donnaient de la profondeur en faisant défiler plusieurs plans à des vitesses différentes : le lointain lentement, le premier plan vite. La MONOCORE-1 a quatre plans, et peut même faire défiler chaque ligne de l'écran différemment (chapitre 10).

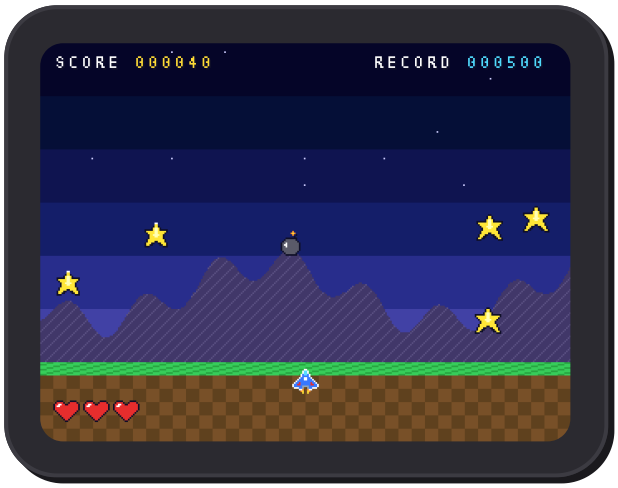
9

Projet : ATTRAPE-ÉTOILES

Un jeu complet : écran titre, score, record, vies, musique et explosions.



L'écran titre.



Une partie en cours.

Le principe : des étoiles et des bombes tombent du ciel. Déplacez le vaisseau avec LEFT et RIGHT pour attraper les étoiles (10 points chacune) et éviter les bombes (une vie perdue). Plus le score monte, plus tout tombe vite. Le jeu tient en 156 lignes, rangées en **fonctions** qui ont chacune un rôle.

Organisation du programme

Fonction	Rôle
`main`	prépare la console, puis enchaîne pour toujours : titre, partie, fin de partie
`title_screen`	écran titre : affiche le titre et les règles, joue la musique, attend START
`game`	la partie : manette, objets, explosion, score, jusqu'à ce qu'il n'y ait plus de vies
`drop_item`	« lâcher un objet » : le place en haut de l'écran, au hasard, étoile ou bombe
`move_items`	fait tomber les objets et teste s'ils touchent le vaisseau
`show_score`	écrit le score et le record, dessine les cœurs
`game_over`	affiche PARTIE TERMINEE pendant trois secondes

Les structures

Chaque objet qui tombe a une position, une vitesse, et peut être une étoile ou une bombe. Plutôt que d'avoir quatre tableaux séparés, on regroupe ces informations dans une **structure** : `item_t` (« objet »). Le tableau `items[6]` contient six objets, et `it->y` désigne la position verticale de l'objet `it`.

Le listing complet

LISTING 07_stars/project.txt

```
# Attrape-Etoiles: the complete game of chapter 9
name    = stars
title   = ATTRAPE-ETOILES
sources = main.c
sprites = sheet.png
maps    = landscape.png
sounds  = boom.wav
```

LISTING 07_stars/main.c

```
// ATTRAPE-ETOILES - the complete game of the MONOCORE-1 manual
#include "easy.h"

extern const uint8_t sheet_chr[], sheet_pal[];
extern const uint8_t landscape_chr[], landscape_pal[], landscape_map[];
extern const uint32_t landscape_width, landscape_height;
extern const int8_t boom_data[];
extern const uint32_t boom_count, boom_rate;

// pictures of the sheet
#define SHIP      0
#define STAR      1
#define STAR2     2
#define BOMB      3
#define EXPLOSION 4
#define HEART     5

// sprites in use
#define SPR_PLAYER  0
#define SPR_ITEM    1      // 1 to 6: the falling items
#define NUM_ITEMS   6
#define SPR_LIFE    10     // 10 to 12: the hearts

typedef struct {
  int active;
  int x, y;             // position in pixels
  int speed;            // pixels per frame
  int bomb;             // 1 = bomb, 0 = star
} item_t;

static item_t items[NUM_ITEMS];
static int player_x, score, best, lives, explosion;

static const char *tune =
  "E G C5 - B A G - A B C5 D5 E5 - - . "
  "E5 D5 C5 - B A G - F A G E D - - . ";

// ------------------------------------------------------------- display
static void show_score(void) {
  print(1, 1, "SCORE");
  print_num(7, 1, score, 6, 1);
  print(25, 1, "RECORD");
  print_num(32, 1, best, 6, 2);
  for (int i = 0; i < 3; i++) {
    if (i < lives) sprite(SPR_LIFE + i, 8 + i * 18, 214, HEART);
    else hide_sprite(SPR_LIFE + i);
  }
}

static void hide_all(void) {
```

```
  for (int n = 0; n < 64; n++) hide_sprite(n);
}

// ------------------------------------------------------------- items
static void drop_item(item_t *it) {
  it->active = 1;
  it->x = 8 + random(288);
  it->y = -16 - random(80);
  it->speed = 1 + random(2) + score / 300;     // faster and faster
  if (it->speed > 6) it->speed = 6;
  it->bomb = random(4) == 0;                   // one chance in four
}

static void move_items(void) {
  for (int i = 0; i < NUM_ITEMS; i++) {
    item_t *it = &items[i];
    it->y = it->y + it->speed;
    if (it->y > 240) drop_item(it);            // out at the bottom: drop it again from
the top

    if (collide(it->x, it->y, player_x, 196)) {
      if (it->bomb) {
        lives = lives - 1;
        explosion = 30;                        // 30 frames of explosion
        play_sample(0, boom_data, boom_count, boom_rate, 230);
      } else {
        score = score + 10;
        tone(1, 1200, 120);                    // a short "ding"
      }
      drop_item(it);
    }
    int picture = it->bomb ? BOMB : (((frame_count / 8 + i) % 2) ? STAR : STAR2);
    sprite(SPR_ITEM + i, it->x, it->y, picture);
  }
}

// ------------------------------------------------------------- screens
static void title_screen(void) {
  cls();
  hide_all();
  print_ink(11, 7, "ATTRAPE-ETOILES", 1);
  print(6, 11, "ATTRAPE LES ETOILES : 10 POINTS");
  print(6, 13, "EVITE LES BOMBES    : -1 VIE");
  print_ink(11, 27, "UN JEU MONOCORE-1", 0);
  print(25, 1, "RECORD");
  print_num(32, 1, best, 6, 2);
  play(2, tune, 10, 90);

  for (;;) {
    wait_frame();
    if ((frame_count / 30) % 2 == 0) print_ink(12, 18, "APPUIE SUR START", 6);
    else                             print    (12, 18, "                ");
    for (int i = 0; i < 4; i++)                // four stars dancing under the rules
      sprite(SPR_ITEM + i, 152 + (i - 2) * 50 + 25, 118 + ((frame_count / 3 + i * 3) %
8), STAR);
    scroll_map(frame_count / 2, 0);
    if (pressed(1, START)) break;
  }
  stop(2);
}

static void game(void) {
  cls();
  hide_all();
  score = 0; lives = 3; explosion = 0;
  player_x = 152;
  for (int i = 0; i < NUM_ITEMS; i++) drop_item(&items[i]);

  while (lives > 0) {
    wait_frame();
    if (held(1, LEFT)  && player_x > 4)   player_x -= 3;
    if (held(1, RIGHT) && player_x < 300) player_x += 3;
    if (frame_count % 8 == 0) quiet(1);        // ends the "ding"

    if (explosion > 0) {
```

```
      explosion--;
      sprite(SPR_PLAYER, player_x, 196, EXPLOSION);
    } else {
      sprite(SPR_PLAYER, player_x, 196, SHIP);
      move_items();                          // the game pauses during the explosion
    }
    if (score > best) best = score;
    show_score();
    scroll_map(frame_count, 0);
  }
}

static void game_over(void) {
  for (int i = 0; i < NUM_ITEMS; i++) hide_sprite(SPR_ITEM + i);
  print_ink(12, 12, "PARTIE TERMINEE", 4);
  for (int t = 0; t < 180; t++) {              // 3 seconds, or START
    wait_frame();
    if (pressed(1, START)) break;
  }
}

int main(void) {
  init();
  background(0, 0, 0);
  load_sprites(sheet_chr, sheet_pal, 8);
  load_map(landscape_chr, landscape_pal, landscape_map, landscape_width,
landscape_height, 9);
  best = 500;
  for (;;) {
    title_screen();
    game();
    game_over();
  }
}
```

Pour aller plus loin

Le jeu est à vous : modifiez-le ! Quelques idées, de la plus simple à la plus ambitieuse :

1. Changez les couleurs, la vitesse, le nombre de points par étoile.
2. Ajoutez un objet rare (le cœur, dessin 5) qui rend une vie.
3. Faites défiler le décor plus vite quand le score augmente.
4. Ajoutez une deuxième voix à la musique (une basse sur le canal 3).
5. Mode deux joueurs : un second vaisseau commandé par la manette 2 (`held(2, LEFT)`).
6. Ajoutez le tir : le bouton 1 lance un projectile qui détruit les bombes.

10

Sous le capot

La bibliothèque easy n'est qu'un raccourci : voici ce que la console sait faire d'autre.

Toutes les fonctions de `easy` écrivent dans des **registres** : des adresses mémoire spéciales qui commandent les circuits de la console. `monocore.h` leur donne des noms. Vous pouvez les utiliser à côté de `easy`, à condition de respecter son rangement (plans 0 et 1, palettes 0 à 15, sprites 0 à 63) ou de ne pas l'utiliser du tout.

Changer la taille de l'écran

```
SCREEN_W = 256;   // from 224 to 384 pixels wide
SCREEN_H = 224;   // from 192 to 288 lines
```

La nouvelle taille prend effet à l'image suivante. Pour un jeu vertical (comme Pac-Man ou Galaga), on choisit 224 x 288 et on demande que l'image soit tournée d'un quart de tour avec `DISP_CTRL |= DISP_ROT(1)` : le simulateur le fait déjà, le core MiSTer le fera dans sa prochaine version.

Les quatre plans

Les plans 2 et 3, inutilisés par `easy`, se règlent comme le décor : `LAYER_CTRL(n)` (taille des tuiles, 4 ou 8 bits par pixel, taille de la carte, priorité), `LAYER_CHR(n)` (adresse des dessins), `LAYER_SCROLLX(n)` et `LAYER_SCROLLY(n)`, et la carte `TILEMAP(n)`. Il faut ensuite les allumer : `DISP_CTRL |= DISP_LAYER(2)`.

Le défilement ligne par ligne

Avec l'option `LC_LINESCROLL`, chaque ligne de l'écran ajoute son propre décalage au défilement du plan : routes de jeux de course, vagues, effets de chaleur.

```
LAYER_CTRL(1) |= LC_LINESCROLL;
for (int y = 0; y < 240; y++) {
  LINESCROLL(1)[y].dx = (y > 120) ? (y - 120) * (frame_count % 64) / 32 : 0;
  LINESCROLL(1)[y].dy = 0;
}
```

Les sprites sans limites

La console gère **512 sprites** (la table `SPRITES[n]`), de 8 x 8 à 256 x 256 pixels (largeur et hauteur en cellules de 8 pixels), en 16 ou 256 couleurs, avec un zoom indépendant en largeur et en hauteur (`zoomx`, `zoomy` : 256 = taille normale). Le registre `SPR_COUNT` dit combien d'entrées la console parcourt.

Les interruptions

Une **interruption** arrête le programme un instant pour exécuter `irq_handler()`. `easy` utilise celle du début de la pause verticale (VBLANK). Il en existe d'autres : LINE (au début d'une ligne choisie par `LINE_CMP`, pour changer un registre en cours d'image), TIMER0 et TIMER1 (à intervalle régulier, en cycles du processeur : `T0_RELOAD = CPU_HZ / 240` pour 240 fois par seconde), DMA (fin de transfert).

Le DMA

Le DMA copie ou remplit la mémoire bien plus vite que le processeur : `dma_copy(destination, source, octets)` et `dma_fill(destination, motif, octets)`. Idéal pour charger une palette, une carte de décor, ou des dessins dans la **CHR RAM** (128 Ko de graphismes modifiables en cours de jeu, à l'adresse `CHRRAM_BASE`).

Le temps du processeur

Le processeur exécute environ **780 000 instructions par image**. C'est énorme : un 68000 de borne d'arcade en exécutait 25 000. Le moniteur du simulateur affiche la charge : si elle dépasse 100 %, le jeu ralentit. Gardez une marge, car le simulateur estime le temps à environ 10 % près.

Règle d'or

Ne comptez jamais sur un nombre de cycles pour régler la vitesse d'un jeu : synchronisez-vous toujours sur les images (`wait_frame()`) ou sur un timer. C'est ce qui garantit que le jeu tourne pareil dans le simulateur et sur la console.

La bibliothèque easy

Toutes les fonctions, d'un coup d'oeil, avec la traduction de leur nom.

Fonction	Traduction	Description
`init()`	initialiser	prépare la console : à appeler une fois, au début
`wait_frame()`	attendre l'image	attend l'image suivante (1/60 de seconde), lit les manettes, fait avancer les mélodies
`frame_count`	nombre d'images	variable : images depuis le démarrage
`random(n)`	au hasard	nombre au hasard de 0 à n - 1
`background(r, g, b)`	fond	couleur du fond (rouge, vert, bleu de 0 à 255)
`ink(n, r, g, b)`	encre	couleur de l'encre de texte n (0 à 7)
`load_palette(n, pal)`	charger une palette	range une palette de 16 couleurs (fichier .pal) en palette n
`cls()`	effacer l'écran	efface tout le texte (de *CLear Screen*)
`print(col, row, text)`	imprimer	écrit en blanc, colonne *col*, ligne *row*
`print_ink(col, row, text, ink)`	imprimer en couleur	écrit avec l'encre choisie
`print_num(col, row, n, digits, ink)`	imprimer un nombre	écrit un nombre sur *digits* chiffres, complété par des zéros
`held(player, button)`	tenu	vrai si le bouton est enfoncé
`pressed(player, button)`	appuyé	vrai si le bouton vient d'être enfoncé
`load_sprites(chr, pal, palette)`	charger les sprites	choisit la planche de sprites
`sprite(n, x, y, picture)`	sprite	affiche le sprite n (0 à 63) avec le dessin *picture*
`hide_sprite(n)`	cacher	cache le sprite n

Fonction	Traduction	Description
`flip_sprite(n, h, v)`	retourner	retourne le sprite n
`scale_sprite(n, percent)`	mettre à l'échelle	agrandit ou réduit le sprite n
`sprite_palette(n, palette)`	palette du sprite	change la palette du sprite n
`collide(x1, y1, x2, y2)`	se heurter	vrai si deux sprites de 16 x 16 se touchent
`load_map(chr, pal, map, w, h, palette)`	charger le décor	installe un décor de w x h tuiles
`scroll_map(x, y)`	faire défiler	fait défiler le décor
`tone(channel, freq, volume)`	note	son carré sur un canal
`noise(channel, volume)`	bruit	bruit sur un canal
`quiet(channel)`	silence	fait taire un canal
`play(channel, tune, frames, volume)`	jouer	joue une mélodie en boucle
`stop(channel)`	arrêter	arrête la mélodie du canal
`play_sample(voice, data, count, rate, volume)`	jouer un échantillon	joue un son enregistré

Boutons : `UP` (haut), `DOWN` (bas), `LEFT` (gauche), `RIGHT` (droite), `B1` à `B6`, `START`, `COIN` (pièce). Encres par défaut : 0 blanc, 1 jaune, 2 cyan, 3 vert, 4 rouge, 5 violet, 6 orange, 7 gris.

LE FICHIER PROJECT.TXT

Réglage	Contenu	Noms créés pour le programme
`name`	nom du fichier produit (`build/name.mnc`)	
`title`	titre inscrit dans la ROM	
`sources`	fichiers C du programme	
`sprites`	planches de sprites (PNG de 16 pixels de large)	`x_chr` `x_pal`
`maps`	décors (PNG, tuiles de 16 x 16)	`x_chr` `x_pal` `x_map` `x_width` `x_height`
`sounds`	sons (WAV)	`x_data` `x_count` `x_rate`

`x` est le nom du fichier sans son extension : `sheet.png` donne `sheet_chr` et `sheet_pal`.

Carte mémoire et registres

Pour programmer la console directement (monocore.h).

Adresse	Taille	Contenu
`0x0000_0000`	32 Mo max	ROM : programme, dessins, sons (lecture seule)
`0x1000_0000`	128 Ko	mémoire de travail : variables, pile
`0x1800_0000`	32 Ko	mémoire sauvegardée (scores, réglages)
`0x2000_0000`	4 Ko	registres vidéo
`0x2001_0000`	8 Ko	palette : 4096 couleurs RGB555
`0x2002_0000`	8 Ko	table des 512 sprites, 16 octets chacun
`0x2004_0000`	64 Ko	cartes des 4 plans, 16 Ko chacune
`0x2008_0000`	8 Ko	défilement par ligne des 4 plans
`0x200C_0000`	128 Ko	CHR RAM : dessins modifiables
`0x3000_0000`	4 Ko	registres son
`0x4000_0000`	4 Ko	manettes, interruptions, timers, DMA

VIDÉO

Registre	Rôle
`DISP_CTRL`	bit 0 affichage, bits 1 à 4 plans 0 à 3, bit 5 sprites, bits 8-9 rotation
`SCREEN_W / SCREEN_H`	taille de l'image (224-384 x 192-288)
`BG_COLOR`	numéro de la couleur de fond dans la palette
`LINE_CMP`	ligne de l'interruption LINE
`VCOUNT / FRAME`	ligne en cours, numéro d'image (lecture)
`SPR_CHR / SPR_COUNT`	adresse des dessins de sprites, nombre d'entrées parcourues
`LAYER_CTRL(n)`	plan n : tuiles 16 (bit 0), 8 bpp (bit 1), largeur et hauteur de carte, défilement par ligne (bit 6), priorité (bits 8-9)
`LAYER_CHR(n)`	adresse des dessins du plan n
`LAYER_SCROLLX/Y(n)`	défilement du plan n

SON

Registre	Rôle
`PCM[v].start / length / loop / step`	voix d'échantillons v (0 à 7) : adresse, longueur, boucle, pas (0x10000 = 48 kHz)
`PCM[v].vol_l / vol_r / format`	volumes, format (0 : 8 bits, 1 : 16 bits, 2 : ADPCM)
`PCM[v].ctrl`	1 lance, 0 arrête
`PSG[c].freq`	fréquence du canal c en 1/16 de hertz
`PSG[c].wave / duty`	forme (carré, triangle, dent de scie, bruit), rapport cyclique
`PSG[c].vol_l / vol_r`	volumes
`MASTER_VOL`	volume général

ENTRÉES, INTERRUPTIONS, TIMERS, DMA

Registre	Rôle
`IN_P1 / IN_P2 / IN_SYS / DIPSW`	manettes, boutons système, DIP switches
`IRQ_STATUS / IRQ_ENABLE`	interruptions en attente (écrire 1 pour acquitter) et autorisées
`T0_RELOAD / T0_CTRL`	timer 0 : période en cycles, marche
`CYCLE_LO / CYCLE_HI`	compteur de cycles du processeur (50 MHz)
`DMA_SRC / DST / LEN / CTRL`	copie (CTRL = 1) ou remplissage (CTRL = 3)
`DEBUG_OUT`	un caractère vers la console de debug du simulateur

Limites et dépannage

Ce que la console ne peut pas faire, et que faire quand rien ne marche.

Limite	Valeur
Sprites par ligne	64, et 1280 pixels de sprites
Couleurs	15 par planche ou par décor (4 bits par pixel), 255 en mode 8 bits
Décor	1024 x 1024 pixels (64 x 64 tuiles de 16)
Mémoire de travail	128 Ko pour les variables et la pile
ROM	32 Mo
Son	8 voix d'échantillons + 8 canaux de notes

MESSAGES ET PROBLÈMES FRÉQUENTS

Symptôme	Cause probable et remède
`error: expected ';'`	il manque un point-virgule à la fin de la ligne indiquée (ou de la précédente)
`undefined symbol: sheet_chr`	l'image n'est pas déclarée dans `project.txt`, ou porte un autre nom
`ERROR: tool not found: clang`	LLVM n'est pas installé ou pas dans le chemin (chapitre 2)
`ERROR: no project.txt file`	la commande n'est pas lancée dans le dossier du programme
écran noir	`init()` oublié, ou le programme n'appelle jamais `wait_frame()`
sprite invisible	planche non chargée, sprite hors de l'écran, ou plus de 64 sprites sur la ligne
couleurs bizarres	plus de 15 couleurs dans l'image : réduisez la palette
le jeu ralentit	trop de calculs par image : regardez la charge dans le moniteur
rien sur le MiSTer	barrette SDRAM absente, ou essayer l'option « Lecture SDRAM : Retardée »

MONOCORE-1 · Manuel du programmeur · édition 1, octobre 2026.

Titres en Press Start 2P (Cody Boisclair), texte en Archivo (Omnibus-Type), listings en IBM Plex Mono : polices sous licence SIL Open Font License.

Les exemples se trouvent dans `manual/examples`. Ce manuel se régénère avec `python3 manual/tools/manual.py`.